Temporal Logics: Property Spec. Languages

CS448/648: A comprehensive exploration of temporal logic systems for specifying and verifying properties of computational systems over time.

Prof. Gourinath Banda, CSE, IIT Indore

Lecture 7

Lecture Contents

Linear Temporal Logic

Properties expressed over single computation paths

Computation Tree Logic

Properties expressed over trees of all possible executions

Behavior, Run, Computation Path

A computation path is a sequence of states starting with an initial state: s_0 s_1 s_2 ... where $R(s_i, s_{i+1})$ is true. This sequence represents the behavior of a system over time.

Also called a "run" or "(computation) path". The trace is the sequence of observable parts of states—the sequence of state labels that can be observed externally.

Safety vs. Liveness Properties

Safety Property

"Something bad must not happen"

Example: System should not crash

Characterized by finite-length error traces

Liveness Property

"Something good must happen"

Example: Every packet sent must be received at its destination

Characterized by infinite-length error traces

Classifying Properties: Practice

Examples

1	2	3
Multi-Processor Cache "More than one processor should have a cache line in write mode" **Classification:** Safety property	Grant Signal "The grant signal must be asserted at some time after the request signal is asserted" **Classification:** Liveness property	Request-Acknowledge "Every request signal must receive an acknowledge and the request should stay asserted until the acknowledge signal is received" **Classification:** Combined safety and liveness

What is Temporal Logic?

Temporal logic is a formal system for specifying properties over time, particularly useful for describing the behavior of finite-state systems.

We begin with **propositional temporal logic** as the foundation. Other variants include real-time temporal logic, metric temporal logic, and signal temporal logic, each suited for different applications.

Atomic State Properties (Labels)

An atomic state property is a Boolean formula over state variables. We represent each unique Boolean formula with a distinct color or name such as p, q, etc.

These labels allow us to identify which properties hold at each state in a computation path.

req

Request signal active

req & !ack

Request active, acknowledge not received

Globally (Always) Operator: G p

The formula **G p** is true for a computation path if property p holds at all states (points of time) along that path.

If G p holds along a path starting at s_0, then p must be true at s_0, s_1, s_2 and all subsequent states infinitely into the future.

Eventually Operator: F p

The formula **F p** is true for a path if property p holds at some state along that path—not necessarily immediately, but eventually. This operator captures the notion that something will happen in the future, though we don't specify exactly when.

Next Operator: X p

The formula **X p** is true along a path starting in state s_i if property p holds in the immediate next state s_{i+1}.

This is the most basic temporal operator, looking exactly one step into the future. If X p holds at state s_2, then p must be true at state s_3.

Nesting Temporal Formulas

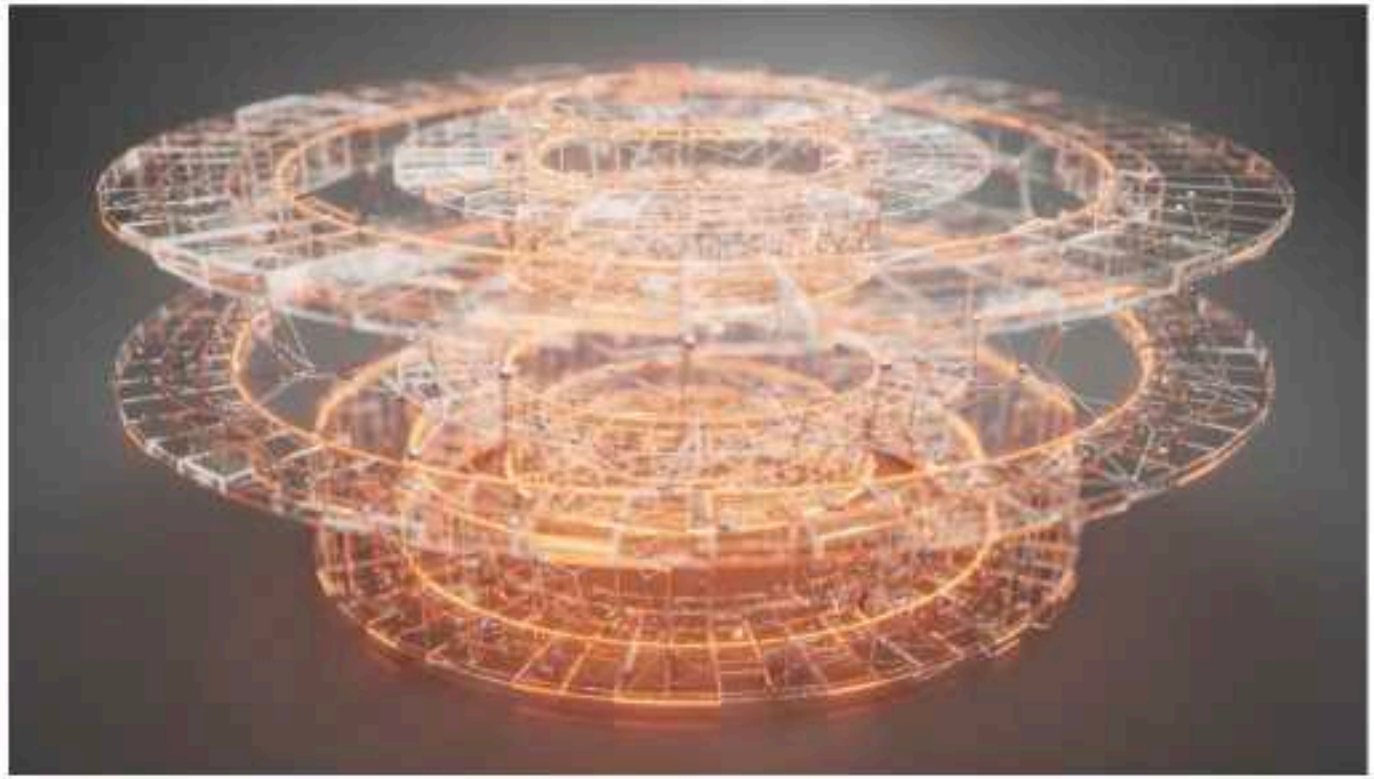
Key Insight

Property p need not be just a Boolean formula—it can be a temporal logic formula itself!

Example

"X p holds for all suffixes of a path"

How do we express this? How do we write formal definitions of Gp, Fp, Xp?



Alternative Notation

In the literature, you'll encounter different symbols for temporal operators. Here's the correspondence:

G (Globally)	F (Eventually)	X (Next)
Also written as □	Also written as ◇	Also written as ○

Practice: Reading Temporal Formulas

Formula 1

G(p → F q)

"Globally, if p holds, then eventually q will hold"

Interpretation: Whenever p becomes true, q must eventually become true afterward.

Formula 2

F(p → (X X q))

"Eventually, if p holds, then q holds two steps later"

Interpretation: At some future point, when p is true, q will be true exactly two states later.

Until Operator: p U q

The formula **p U q** is true along a path starting at state s if:

- Property q is true in some state reachable from s
- Property p is true in all states from s until q holds

This operator captures the notion of one property holding continuously until another property becomes true.

Relationships Between Temporal Operators

Expressiveness Questions

The temporal operators G, F, X, and U are related. Consider these questions:

- Can you express G p purely in terms of F, p, and Boolean operators?
- How about G and F in terms of U and Boolean operators?
- What about X in terms of G, F, U, and Boolean operators?

Key Insight

Some operators can be defined in terms of others, revealing the fundamental expressiveness of the logic.

Expressing Properties in Temporal Logic

01

Multi-Processor Cache Safety

Safety: "More than one processor should have a cache line in write mode"

Formula: $G(\neg(wr_1 \wedge wr_2))$

where wr_1/wr_2 are true if processor 1/2 has the line in write mode

02

Grant After Request

"The grant signal must be asserted at some time after the request signal is asserted"

Formula: G(req → F grant)

03

Request-Acknowledge Protocol

Every request signal must receive an acknowledge and the request should stay asserted until the acknowledge signal is received"

Formula: G(req → (req U ack))

Linear Temporal Logic (LTL)

What we've explored so far are properties expressed over a single computation path or run. This is **Linear Temporal Logic (LTL)**.

LTL reasons about individual executions of a system, treating time as a linear sequence of states extending infinitely into the future.

Temporal Logic Flavors

Two Main Approaches

Linear Temporal Logic

Properties over single paths

Computation Tree Logic

Properties over trees of all possible executions

Where does this "tree" come from? It emerges from the state transition graph of the system, where each state may have multiple possible successor states.

From State Graph to Computation Tree

Kripke Structure

A labeled state transition graph represents all possible states and transitions of a system.

States are labeled with atomic propositions (like p, q, r) that hold in those states.

By "unrolling" this graph from an initial state, we generate an infinite computation tree showing all possible execution paths.

Computation Tree Logic (CTL*)

CTL* introduces two new operators called **path quantifiers** that hold in states (not paths):

A (All paths)	E (Exists a path)
A p: Property p holds along all computation paths starting from the current state	**E p:** Property p holds along at least one path starting from the current state

Example: "The grant signal must always be asserted some time after the request signal is asserted"

Formula: A G (req → A F grant)

Computation Tree Logic (CTL)

Syntactic Restriction

In CTL, every temporal operator (F, G, X, U) must be immediately preceded by either A or E.

For example, you **cannot** write A(FG p) in CTL, but you **can** write A F A G p.

LTL vs CTL

LTL is like having an implicit "A" on the outside of every formula.

CTL as an Approximation of LTL

Weaker Approximation

Useful for finding bugs: if AG EF p fails, then GF p definitely fails.

AG EF p is weaker than **GF p**

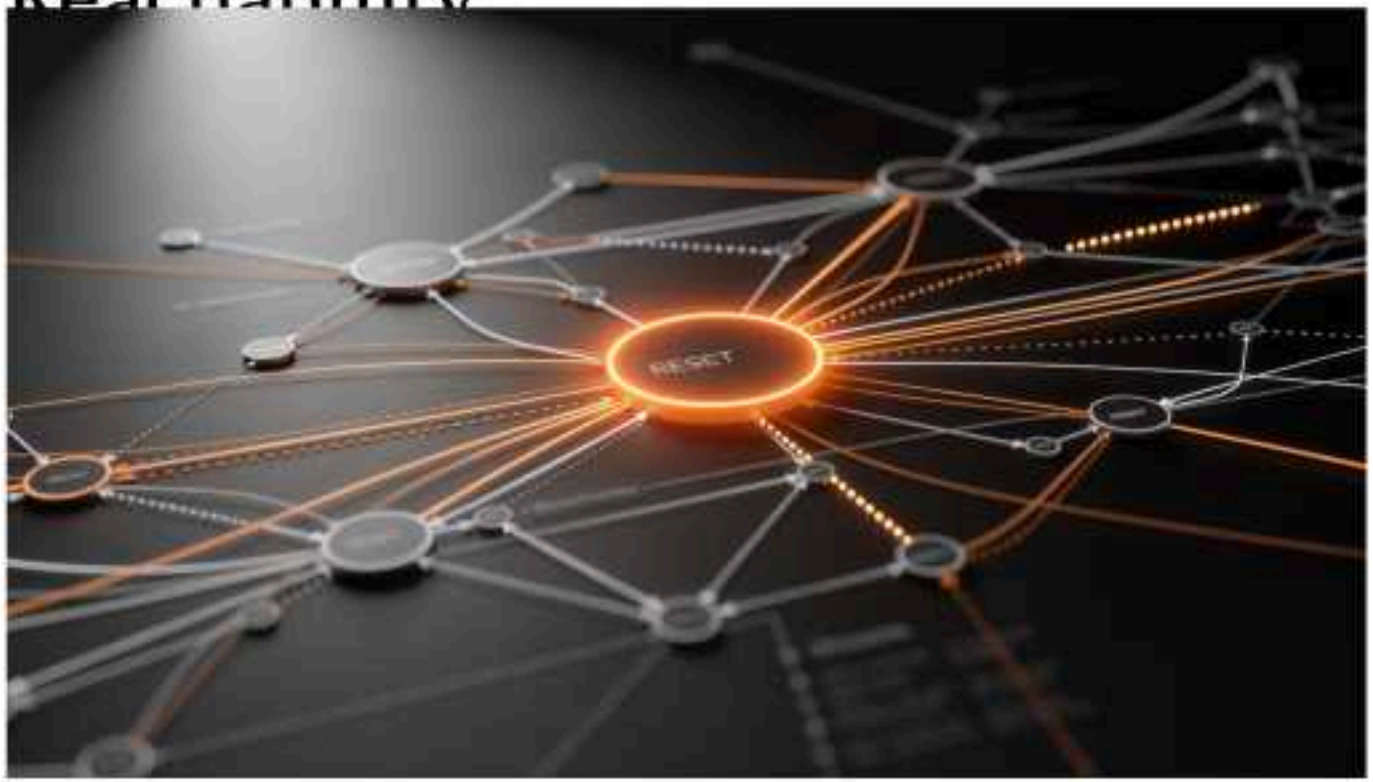
AF AG p is stronger than **FG p**

Stronger Approximation

Useful for verifying correctness: if AF AG p holds, we have stronger guarantees than FG p.

Why is this approximation useful? It allows us to use efficient CTL model checking algorithms while still reasoning about LTL-like properties.

CTL Example: Reachability



Property

"From any state, it is possible to get to the reset state along some path"

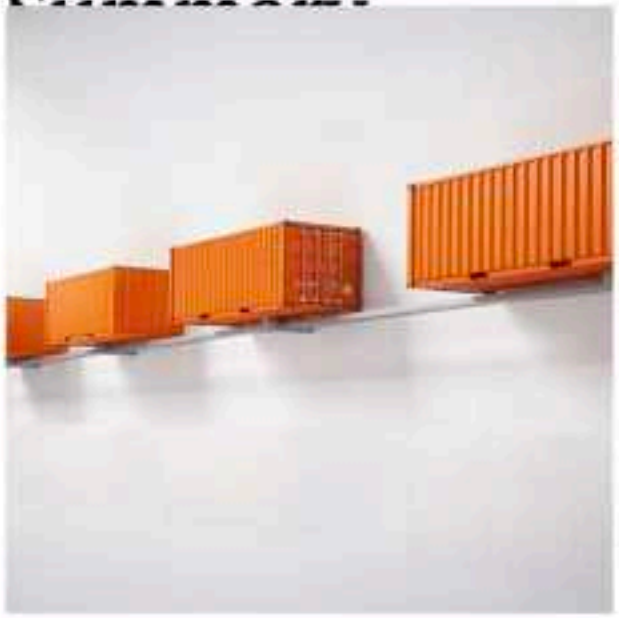
CTL Formula:

AG (EF reset)

This states that globally (in all states), there exists a path to a state where reset holds.

CTL vs. LTL:

Summary



Linear Temporal Logic

Logic for people to understand and use in property specification

Reasons about individual execution traces

Computation Tree Logic

Efficient model checking algorithms available

Reasons about all possible executions simultaneously

Both logics have different expressive powers—some properties can be expressed in one but not the other. Overall, LTL is more commonly used in practice due to its intuitive nature.

Deadlock and Temporal Logic

A Critical Property

Deadlock is an oft-cited property, especially for distributed and concurrent systems. Can you express absence of deadlock as:

- A property of the state graph (graph of all reachable states)?
- A CTL property?
- An LTL property?

Key Observations

The vast majority of properties are safety properties. Liveness properties are useful abstractions of more complicated safety properties, such as real-time response constraints.